

Universidade Federal do Rio Grande – FURG
Centro de Ciencias Computacionais
Sistemas para Internet 2

Web Proxy HTTP com Controle de Conteudo

Relatorio Tecnico

Autor: Andrey Vinicius Santos Souza – 164402

Professor: Prof. Dr. Andre Prisco Vargas

Semestre: 2026/1

Rio Grande, RS – Brasil

2026

Sumário

1	Introducao	2
2	Justificativa da tecnologia	2
3	Organizacao do projeto	2
4	Deciso es de implementacao	3
5	Trechos principais da implementacao	3
6	Dificuldades e aprendizados	3
7	HTTPS e limitacoes do filtro	4
8	Uso no navegador	4
9	Testes realizados	4
10	Uso de inteligencia artificial	5
11	Conclusao	5

1 Introducao

Este relatorio descreve a implementacao de um proxy HTTP didatico com controle de conteudo. O sistema recebe uma URL pelo navegador, consulta uma lista local de dominios bloqueados, busca a pagina original quando o acesso e permitido, aplica substituicoes em respostas HTML e registra cada acesso em arquivo de log.

O objetivo nao foi criar um proxy corporativo completo, mas construir uma versao funcional e explicavel dos mecanismos pedidos no enunciado: repasse, bloqueio, filtragem, registro de acesso e discussao sobre HTTPS.

2 Justificativa da tecnologia

A implementacao usa Python 3 com Flask. A escolha foi feita porque o Flask permite criar rapidamente uma rota HTTP e concentrar o trabalho na logica do proxy: validacao da URL, bloqueio, repasse, filtro e registro de acessos. Para buscar o servidor de origem foi usada a biblioteca `requests`, que simplifica timeout, redirecionamentos e leitura de cabecalhos.

A alternativa com sockets TCP daria mais controle, mas tambem exigiria implementar manualmente o parsing de requisicoes e respostas HTTP. Para o objetivo do trabalho, Flask e `requests` deram um equilibrio melhor entre clareza, funcionamento e facilidade de manutencao.

3 Organizacao do projeto

O projeto foi dividido em modulos pequenos, cada um com uma responsabilidade principal:

- `proxy_controller.py`: recebe a requisicao Flask, valida a URL e coordena o fluxo.
- `content_policy.py`: carrega a lista de dominios bloqueados e decide se o acesso deve ser barrado.
- `origin_client.py`: faz a requisicao ao servidor de origem usando `requests`.
- `text_rewriter.py`: aplica as substituicoes configuradas em respostas HTML.
- `access_audit.py`: grava o log incremental em JSON Lines.
- `html_pages.py`: gera as paginas HTML de erro e bloqueio.

Essa organizacao evita concentrar toda a logica em um unico arquivo e facilita explicar o caminho de uma requisicao durante a apresentacao.

Navegador → rota Flask → politica de bloqueio → cliente HTTP
→ filtro de HTML → gravacao do log → resposta ao navegador

Figura 1: Fluxo interno de uma requisicao HTTP no proxy.

4 Decisoões de implementação

A lista de domínios bloqueados fica em `settings/blocked.json`, usando a chave `bloqueados`. Antes da comparação, o domínio é normalizado para minúsculas, sem porta e sem o prefixo `www..`. Assim, `www.example.com:80` e `example.com` são tratados como o mesmo destino para fins de bloqueio.

As substituições ficam em `settings/words.json`. O filtro é aplicado apenas quando o cabeçalho `Content-Type` indica `text/html`. Isso evita tentar alterar imagens, arquivos binários ou respostas JSON. Os termos são ordenados por tamanho decrescente para que frases maiores sejam avaliadas antes de palavras menores.

O proxy também trata query strings. Por exemplo, uma chamada com parâmetro `curso=si2` deve repassar esse parâmetro para o servidor de origem.

O log foi gravado em JSON Lines, no arquivo `storage/access_log.jsonl`. Cada linha é um objeto independente, contendo data, URL, domínio, ação tomada, código de status e quantidade de substituições. Esse formato é adequado para log incremental porque não é necessário reescrever um grande array JSON a cada acesso.

5 Trechos principais da implementação

O controlador central valida a URL e assume HTTP quando o usuário não informa o esquema:

```
if "://" not in target:
    target = f"http://{target}"
if parsed.scheme != "http":
    raise ValueError("Este proxy aceita apenas URLs HTTP.")
```

A política de bloqueio normaliza o domínio antes da comparação:

```
if clean.startswith("www."):
    clean = clean[4:]
return clean in blocked_domains
```

O filtro de texto usa substituição case-insensitive e informa quantas trocas foram feitas:

```
result, count = rule.pattern.subn(rule.replacement, result)
total_replacements += count
```

6 Dificuldades e aprendizados

Uma dificuldade foi entender que o proxy não deve tratar todos os conteúdos como texto. Algumas respostas podem ser imagens, CSS, JavaScript, JSON ou arquivos compactados. Por isso o código preserva o corpo original quando a resposta não é HTML.

Outro ponto importante foi o cabeçalho de codificação. Para facilitar a reescrita de HTML, o proxy pede ao servidor de origem uma resposta sem compressão usando o cabeçalho `Accept-Encoding` com valor `identity`. Isso evita aplicar expressões regulares sobre bytes compactados.

O trabalho reforçou a diferença entre cabeçalhos e corpo da resposta, a importância do `Content-Type`, o papel do domínio na decisão de bloqueio e o cuidado necessário ao modificar uma resposta antes de devolvê-la ao navegador.

7 HTTPS e limitações do filtro

O filtro de conteúdo não funciona em HTTPS comum porque o conteúdo da página é protegido por TLS. Quando um navegador acessa um site HTTPS por proxy, o comportamento esperado é criar um túnel com o método `CONNECT`. Depois disso, o proxy apenas encaminha bytes cifrados entre cliente e servidor.

Como o proxy não possui as chaves da conexão TLS, ele não consegue ler o HTML em texto claro e, portanto, não consegue aplicar substituições. Para filtrar HTTPS seria necessário implementar uma interceptação do tipo Man-in-the-Middle, criando uma autoridade certificadora local, instalando essa CA no navegador e gerando certificados dinâmicos para os sites acessados.

Essa solução é mais complexa, tem implicações de segurança e não faz parte desta entrega. Por isso, este projeto se limita a HTTP e explica o comportamento de HTTPS no relatório, conforme pedido no enunciado.

8 Uso no navegador

O proxy pode ser configurado no navegador como proxy HTTP usando host `127.0.0.1` e porta `5000`. Nesse modo, o navegador envia requisições HTTP para o servidor Flask e o proxy aplica as mesmas regras de bloqueio, repasse, filtro e log.

No Windows, essa configuração fica em *Settings, Network & internet, Proxy, Manual proxy setup*. Basta ativar a opção de servidor proxy e preencher endereço `127.0.0.1` e porta `5000`.

Essa configuração deve ser usada com sites HTTP, como `http://neverssl.com` ou `http://info.cern.ch`. Para HTTPS, o navegador tenta usar `CONNECT`; como esse método não foi implementado nesta versão, o suporte a HTTPS fica fora do escopo.

9 Testes realizados

Foram criados testes unitários com `pytest` para três partes principais:

- política de bloqueio, incluindo domínio com porta e prefixo `www.`;
- filtro de texto, incluindo substituição case-insensitive e preferência por frases maiores;
- validação de URL, incluindo URL sem esquema, esquema inválido e query string.

Também foi feito teste manual com servidor HTTP local para confirmar o repasse e a substituição de termo no HTML.

10 Uso de inteligencia artificial

Usei inteligencia artificial como apoio durante o desenvolvimento do trabalho, principalmente para organizar a arquitetura do projeto, revisar textos do README e deste relatório, criar testes simples com `pytest` e entender como o proxy poderia funcionar quando configurado diretamente no navegador pelo proxy manual do Windows.

As sugestões geradas foram revisadas e testadas antes de entrar na versão final. Eu conferi o funcionamento das partes principais: carregamento dos arquivos `blocked.json` e `words.json`, bloqueio de domínios, filtro de palavras em HTML, gravação do log em JSON Lines, validação das URLs e suporte a requisições HTTP enviadas pelo navegador.

Com esse processo, aprendi melhor a diferença entre acessar o proxy pelo formato didático `localhost:5000/http://site` e configurar o navegador para enviar requisições HTTP pelo proxy. Também revisei a limitação do HTTPS e o motivo de o método `CONNECT` ser necessário para tunelamento TLS.

11 Conclusão

O projeto mostra, de forma prática, que o HTTP permite que um intermediário leia e modifique respostas quando não há criptografia. Também evidencia por que o HTTPS é importante: ele impede que proxies comuns inspecionem e alterem o conteúdo trafegado entre navegador e servidor.

A implementação final cumpre os requisitos obrigatórios do enunciado e usa uma organização modular para deixar as decisões técnicas mais claras.